Preliminary Analysis of Developer Activity and Language Ecosystems in a Large GitHub Follow Network

March 15, 2026

Abstract

This report presents preliminary findings from a large-scale observational study of developer activity within a GitHub follow network. Using the GitHub GraphQL API, we collect activity metadata for accounts followed by a single user whose follow graph exceeds one million accounts. Each account is classified as active or inactive based on the recency of commits to public repositories, and the dominant programming language of the most recently updated repository is recorded.

Analysis of the first $\sim 30{,}000$ accounts reveals systematic differences in activity levels across programming language ecosystems, as well as evidence for a characteristic "half-life" of developer activity on GitHub of approximately ten years. These results provide an empirical snapshot of the structure and longevity of open source developer communities.

1 Introduction

GitHub has become one of the central platforms for collaborative software development, and its public data provides a unique opportunity to study the dynamics of programming language ecosystems and developer activity over time.

Most existing studies rely on repository-level datasets such as GitHub Archive or BigQuery mirrors of repository metadata. While these datasets capture repository events in detail, they provide only indirect insight into the life cycles of individual developers.

The present study instead analyzes a *developer-centered dataset*. Starting from a single GitHub account that follows more than one million other accounts, we crawl the follow network using the GitHub GraphQL API and collect basic metadata describing each followed account. The goal is to estimate:

- the fraction of developers who remain active over time,
- differences in activity rates across programming languages,
- and the approximate lifetime of developer participation on GitHub.

The results reported here are preliminary and based on the first $\sim$ 30,000 accounts processed during the crawl.

2 Dataset Construction

The dataset is collected using the GitHub GraphQL API. For each followed account we retrieve:

- account creation timestamp,
- number of followers,
- number of public repositories,
- the most recently pushed public repository,
- and the primary language of that repository.

Accounts are classified as **active** if the most recent repository push occurred within the previous year; otherwise the account is classified as **inactive**. Accounts for which no language-tagged repository is available appear in the dataset under the label None.

For the first $N = 30{,}182$ accounts processed, the resulting dataset contains:

Active accounts	18,007
Inactive accounts	12,175
Active fraction	≈ 0.60

Because the sample is drawn from a follow network rather than uniformly across GitHub, it is expected to be biased toward more active developers.

3 Language Activity Ratios

For each programming language we compute an *activity ratio*

$$R = \frac{\text{active}}{\text{active} + \text{inactive}}.$$

This metric estimates the probability that a developer associated with a given language ecosystem is still actively committing code. Table 1 shows the complete results for all languages with at least 59 tagged accounts in the dataset.

Language	Active	Inactive	Total	Activity Ratio
None	3041	5155	8196	0.37
Astro	54	8	62	0.87

continued on next page

Language	Active	Inactive	Total	Activity Ratio
Rust	423	67	490	0.86
TypeScript	2480	450	2930	0.85
Dart	151	38	189	0.80
Lua	114	26	140	0.81
TeX	65	21	86	0.76
Python	3366	1259	4625	0.73
SCSS	92	38	130	0.71
HTML	1386	599	1985	0.70
Go	484	203	687	0.70
PowerShell	43	18	61	0.70
R	133	62	195	0.68
Shell	415	208	623	0.67
Kotlin	105	54	159	0.66
Swift	125	66	191	0.65
JavaScript	1695	954	2649	0.64
Jupyter Notebook	695	383	1078	0.64
C#	277	192	469	0.59
C++	476	346	822	0.58
MATLAB	43	32	75	0.57
Dockerfile	34	27	61	0.56
CSS	228	200	428	0.53
Vue	92	83	175	0.53
PHP	188	174	362	0.52
Java	507	527	1034	0.49
Ruby	111	120	231	0.48
Vim Script	27	33	60	0.45
C	307	400	707	0.43
Objective-C	9	50	59	0.15

Table 1: Activity ratios for all languages with at least 59 accounts. Languages are ordered by activity ratio descending.

Several patterns are immediately visible.

Newer language ecosystems such as Rust, TypeScript, and Astro exhibit the highest activity ratios, indicating strong ongoing development momentum. Astro (0.87), in particular, reflects the recency of its adoption: the framework was released publicly in 2021 and its community consists almost entirely of developers who joined or began using GitHub recently. Rust (0.86) and TypeScript (0.85) similarly attract developers who are active in contemporary engineering contexts.

At the other end of the spectrum, older ecosystems show markedly lower retention. Java (0.49) and Ruby (0.48) both fall below the overall mean, reflecting the large populations of historical accounts accumulated since their peaks of adoption. The None category—accounts for which no language-tagged repository is available—

shows the lowest ratio among interpretable entries (0.37), consistent with the expectation that inactive accounts are less likely to have recent, language-tagged repositories.

Objective-C (0.15) is an outlier in the full distribution and warrants separate attention; it is discussed in Section 3.1.

3.1 The Objective-C Signal

The extremely low activity ratio for Objective-C ($R = 0.15$, 9 active out of 59 total) stands apart from all other ecosystems in the dataset. This figure is consistent with the history of Apple platform development: following the introduction of Swift at WWDC 2014 and its open-sourcing in 2015, the iOS and macOS developer communities migrated wholesale to the new language over the subsequent three to five years. Developers whose most recent language-tagged repository is still Objective-C are, by definition, those who have not yet updated to Swift—a population skewed heavily toward inactivity. The ratio thus functions here not merely as a measure of ecosystem vitality but as a fossil record of a completed platform migration.

4 Account Age and Developer Survival

Using account creation timestamps and most recent commit timestamps, we estimate the fraction of developers remaining active as a function of account age.

Account Age (years)	Active Fraction
0	0.662
5	0.670
10	0.500
11	0.478

The curve crosses 0.5 at approximately ten years, suggesting a *developer activity half-life* of roughly a decade. This implies that half of the developers who create GitHub accounts will cease making public commits within approximately ten years of joining the platform.

The survival fraction is relatively stable from account ages zero to five years before beginning its decline, which suggests that early dropout is modest and that the primary attrition occurs in later years. This is consistent with developers who remain active throughout the early stages of their careers but gradually reduce open source contribution as professional and other commitments evolve.

5 Language-Specific Developer Lifetimes

We also compute median observed developer lifetimes by language, defined as the interval between account creation and most recent repository activity. Table 2 presents the full results.

Language	Median Lifetime (years)	n
Elixir	12.92	32
Clojure	12.13	52
Emacs Lisp	12.03	38
Perl	11.76	54
Nix	11.46	41
Vim Script	10.11	60
Lua	10.04	140
Common Lisp	9.50	23
Julia	9.28	50
Swift	9.02	191
Ruby	8.85	231
SCSS	8.66	130
Makefile	8.66	44
Shell	8.62	623
Dockerfile	8.54	61
Go	8.35	687
Rust	8.30	490
TeX	7.93	86
Scala	7.69	35
HCL	7.37	52

Table 2: Median developer lifetimes by language (languages with $n \geq 23$, ordered by median lifetime descending).

These estimates combine two effects: the maturity of the ecosystem and the retention of developers within that ecosystem. The languages at the top of Table 2—Elixir, Clojure, Emacs Lisp, Perl, and Nix—are either niche communities with highly committed long-term practitioners or languages that enjoyed their peak adoption in the early years of GitHub. Older accounts that remain active have accumulated long measured lifetimes by construction.

The contrast between median lifetime and activity ratio illuminates different aspects of ecosystem health. Elixir exhibits a long median lifetime (12.92 years) but does not appear among the highest activity ratios, suggesting that its developer base is experienced and long-tenured but not rapidly growing. Conversely, Rust exhibits a relatively modest median lifetime (8.30 years) while maintaining a very high activity ratio (0.86), consistent with a younger but highly engaged and growing community.

6 Discussion

Even from this partial dataset several structural features of the modern software ecosystem are legible.

First, newer programming languages such as Rust and TypeScript appear to have exceptionally high developer retention rates, while ecosystems associated with earlier

waves of GitHub adoption carry larger inactive populations. Second, developer participation on GitHub follows a characteristic decay pattern with a half-life of roughly ten years, suggesting that the platform's active developer population turns over substantially on a decadal timescale. Third, the distinction between median lifetime and activity ratio reveals that ecosystem maturity and community vitality are not equivalent: a community can be old and committed (Elixir, Clojure) or young and growing (Rust, TypeScript) with importantly different implications for its future trajectory.

The NONE category deserves further investigation in subsequent work. Its low activity ratio (0.37) relative to the overall mean (0.60) is consistent with a hypothesis that inactive developers are less likely to maintain language-tagged repositories, but it may also reflect structural features of the follow network itself—for example, spam or bot accounts, which tend to lack repository content.

7 Future Work

The present analysis is based on the first $\sim$ 30,000 accounts processed by the crawler. The full dataset is expected to contain approximately one million accounts. Completion of the crawl will enable more precise estimation of:

- developer survival curves disaggregated by language and account vintage,
- language ecosystem migration patterns detectable from sequential account activity,
- the relationship between developer influence (followers) and long-term activity,
- and formal survival models (e.g., Weibull or log-normal fits) to characterize the attrition process quantitatively.

Because the dataset is developer-centered rather than repository-centered, it provides a complementary perspective on the dynamics of open source communities not available from existing large-scale GitHub datasets.

8 Conclusion

Preliminary analysis of a large GitHub follow network suggests that developer activity on the platform exhibits a characteristic lifetime on the order of a decade. Activity ratios vary significantly across programming language ecosystems: Astro, Rust, and TypeScript lead with ratios above 0.85, while Objective-C, Java, and Ruby trail with ratios at or below 0.49. Median developer lifetimes are longest in older niche communities (Elixir, Clojure, Emacs Lisp) and shorter in newer, rapidly growing ecosystems.

Completion of the full dataset will allow a more detailed empirical picture of developer lifecycles and language ecosystem evolution on GitHub, including formal survival modeling and analysis of cross-language migration trajectories.